

ThinkBeforeClick

Architecture Reference — purpose of every folder and file in the project structure

Project Root

thinkbeforeclick/	Root folder containing the entire project: backend, web app, browser extension, and docs.
README.md	Top-level overview of the project for anyone opening the repo for the first time.

backend/ — FastAPI service (the shared brain for web app + extension)

requirements.txt	Pinned list of Python packages needed to run the backend (fastapi, uvicorn, sqlalchemy, etc.).
.env	Stores secret values (API keys like Google Safe Browsing) outside of source code; excluded from git.
thinkbeforeclick.db	The actual SQLite database file created on first run; holds scans and reports data.

app/

main.py	Entry point that creates the FastAPI app, registers all routes, and creates DB tables on startup.
----------------	---

app/api/

__init__.py	Marks this folder as a Python package so its modules can be imported.
health.py	Defines GET /health, a simple check to confirm the server is alive.
analyze.py	Defines POST /analyze, the main endpoint that runs the detection pipeline on a message/URL.
report.py	Defines POST /report, lets a user flag a result so it can feed future ML retraining.

app/schemas/

__init__.py	Marks this folder as a Python package.
analyze_schema.py	Pydantic models defining the exact shape/validation of analyze request and response data.

app/services/

__init__.py	Marks this folder as a Python package.
rule_engine.py	Phase 2: regex/keyword-based detection logic for suspicious messages and URLs.
ml_engine.py	Phase 3: wraps the trained TF-IDF + Logistic Regression model as a callable scoring function.
threat_intel.py	Phase 4: calls external services (Google Safe Browsing, WHOIS) to verify URL reputation/age.

fusion_engine.py	Phase 5: combines rule, ML, and threat-intel scores into one final risk verdict + explanation.
-------------------------	--

app/ml/

train_model.py	Standalone script (Phase 3) that trains the classifier from the prepared dataset.
dataset/	Holds the cleaned, labeled public datasets (SMS spam + phishing email data) used for training.
model/classifier.joblib	The saved, trained ML model file loaded by ml_engine.py at runtime.

app/db/

__init__.py	Marks this folder as a Python package.
database.py	Sets up the SQLAlchemy engine and session used to talk to the SQLite database.
models.py	Defines the Scan and Report table structures (columns, types) stored in SQLite.

app/core/

config.py	Central place for app settings, environment variables, and API key loading.
logging.py	Configures how and where the application logs messages/errors.

Note: services/ (beyond rule_engine.py), ml/, and core/ are placeholders reserved for later phases — intentionally empty until their phase begins.

webapp/ — Public-facing website (Phase 6)

index.html	The main analyzer page where a student pastes a message/URL to check.
how-it-works.html	Plain-language explanation of the 3-layer detection pipeline, for transparency/trust.

static/css/

style.css	Visual styling for the web app pages.
------------------	---------------------------------------

static/js/

analyzer.js	Front-end script that calls the backend's /analyze endpoint and renders the result.
report.js	Front-end script that calls the backend's /report endpoint when a user flags a result.

assets/

icons/	Image assets (icons, logos) used across the web app pages.
---------------	--

extension/ — Browser extension (Phases 8-9)

manifest.json	Manifest V3 configuration: permissions, scripts, and metadata that define the extension.
background.js	Service worker running in the background; manages the badge count and right-click context menu.

content-script.js

Phase 8: runs quietly on webmail/webchat pages doing local-only rule-based scanning (Tier 1).

popup/

popup.html

Phase 9: the deep-check popup UI, same analyzer layout as the web app (Tier 2).

popup.css

Styling for the popup UI.

popup.js

Calls the deployed backend's /analyze endpoint when the popup is used for a full check.

icons/

icon16.png /

icon48.png /

icon128.png

Extension icons at the sizes Chrome requires for toolbar, management page, and store listing.

docs/ — Planning documentation

**ThinkBeforeClick_Bluepr
int.md**

The original planning document: idea, architecture diagrams, phase breakdown, confirmed decisions.